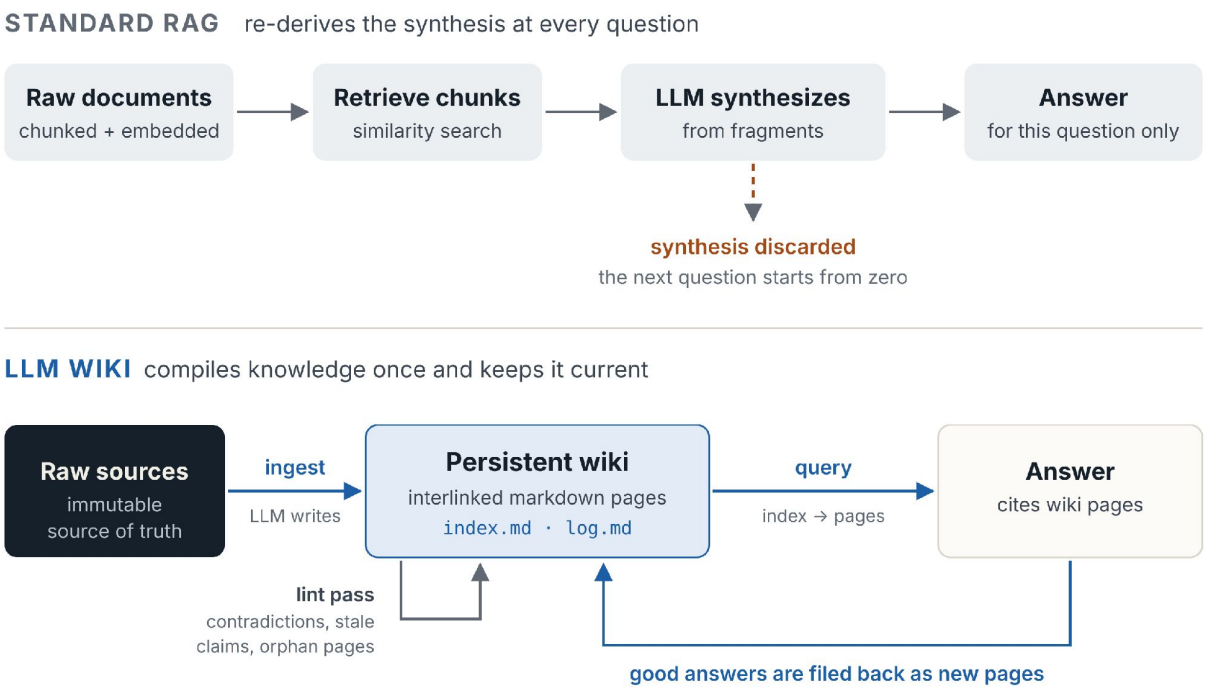
PART IV · 40 MINUTES

Case studies

What does the never-ending perspective change for a simple NLP task — and how do current systems realize it?

Karpathy's LLM Wiki: continuously growing a KB for QA



Human: curates sources, explores, asks questions

LLM: summarizing, cross-linking, filing and logging, the bookkeeping

Figure generated with help from Claude Code

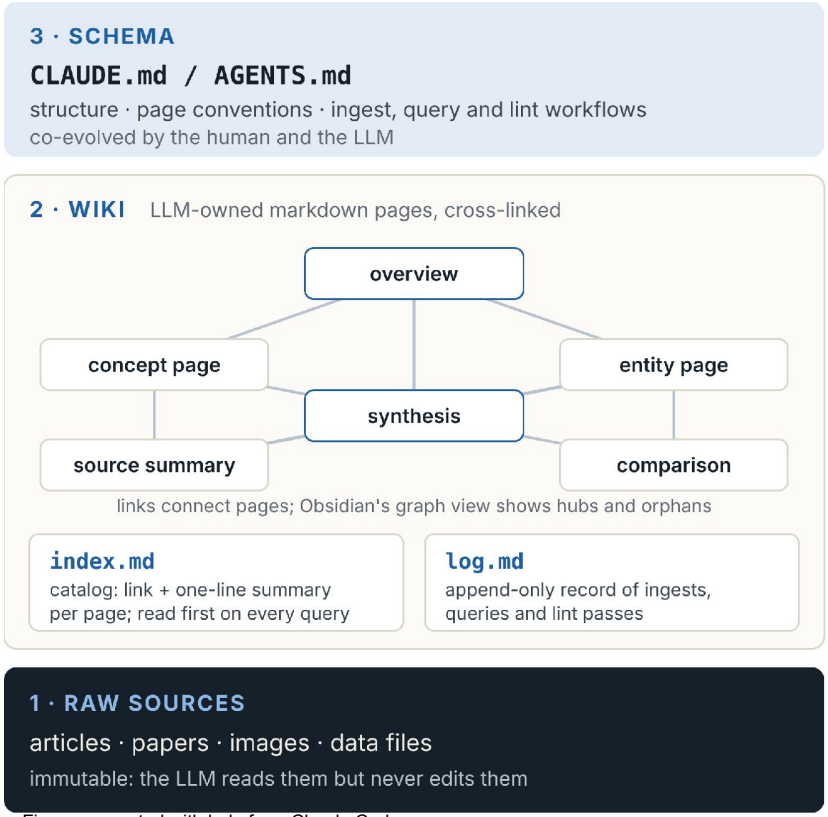
Karpathy's LLM Wiki: continuously growing a KB for QA

- Instead of retrieving raw chunks at every question, an LLM agent incrementally builds and maintains a KB (wiki) of interlinked markdown pages that sits between the user and the sources.
- The memory is structured as a wiki (md files), which can be handled by coding agents. It becomes "a persistent, compounding artifact."
- The human curates sources, explores and asks questions. The LLM does the bookkeeping: summarizing, cross-linking, filing and logging.
- The wiki is learned state that lives outside the model weights, and it stays human-readable, versioned and reversible.

Source: A. Karpathy, "LLM Wiki", GitHub gist, April 2026 (gist.github.com/karpathy/442a6bf555914893e9891c11519de94f).

Karpathy's LLM Wiki: continuously growing a KB for QA

KB Representation



Example layout

illustrative: the gist prescribes none

```
my-wiki/  (git repo)
├─ CLAUDE.md
├─ raw/
│  ├─ articles/
│  ├─ papers/
│  └─ assets/
└─ wiki/
   ├─ index.md
   ├─ log.md
   ├─ overview.md
   ├─ concepts/
   ├─ entities/
   ├─ sources/
   └─ analyses/
```

Git: every edit is a diff that can be reviewed or reverted

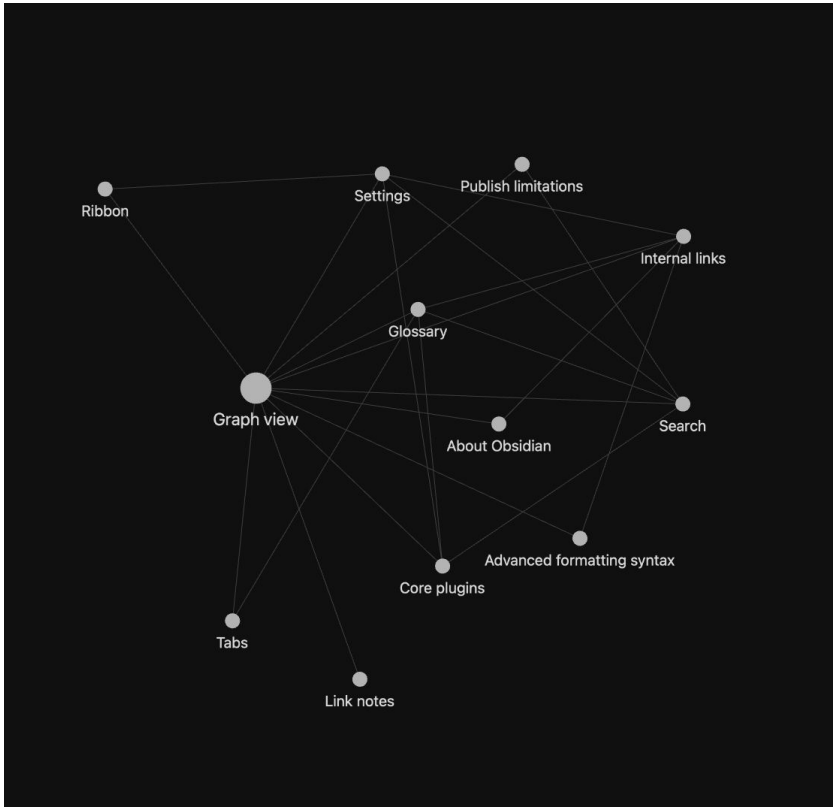
Obsidian: browse pages, backlinks and the link graph

Figure generated with help from Claude Code

Karpathy’s LLM Wiki: continuously growing a KB for QA

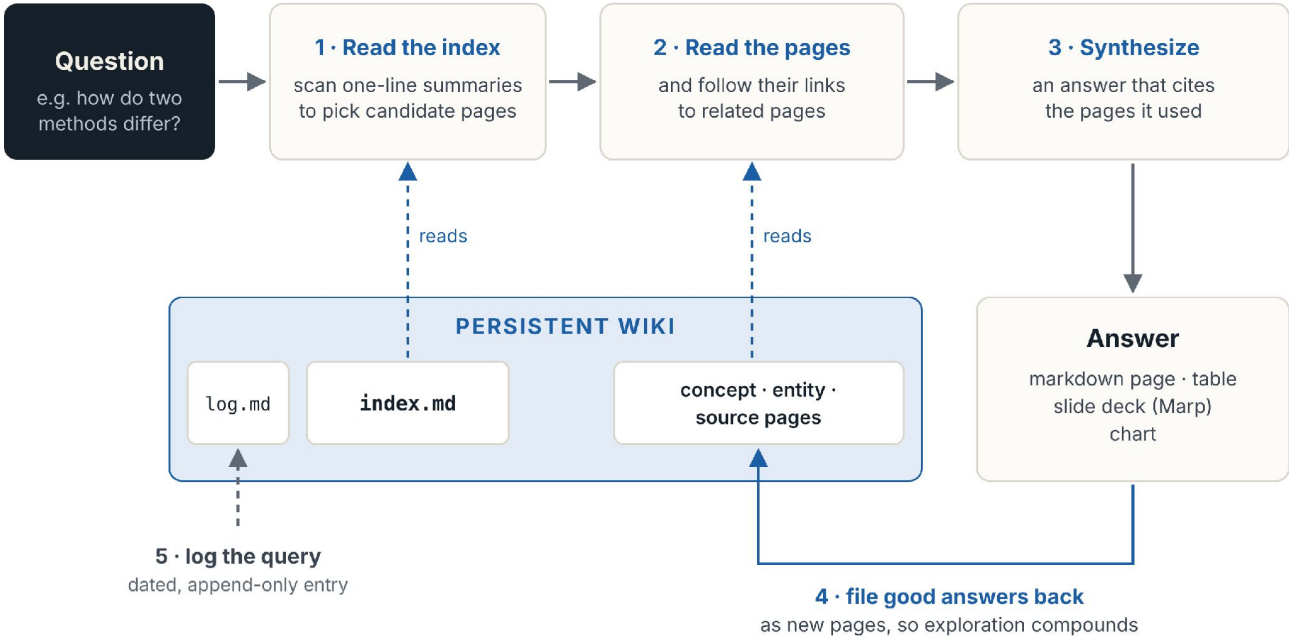
KB Representation

Source: https://obsidian.md/help/plugins/graph



Karpathy's LLM Wiki: continuously growing a KB for QA

The QA Task



At scale: index-only navigation works for roughly 100 sources and hundreds of pages.
Beyond that, add search, e.g. qmd: hybrid BM25 + vector retrieval with LLM re-ranking.

Figure generated with help from Claude Code

Source: A. Karpathy, "LLM Wiki", GitHub gist, April 2026
(gist.github.com/karpathy/442a6bf555914893e9891c11519de94f).

Karpathy's LLM Wiki: continuously growing a KB for QA

The QA Task

- The agent reads index.md first, opens the pages that look relevant, follows their links, and writes an answer that cites the pages it used.
- The output can be a markdown page, a comparison table, a Marp slide deck or a chart.
- "Good" answers, such as comparisons or newly found connections, are filed back as new pages instead of being lost in the chat. Every query is also appended to log.md.
- The index alone suffices for about 100 sources and hundreds of pages. Beyond that, add a search tool such as qmd (hybrid BM25 and vector search with LLM re-ranking).
- The costly synthesis moves from query time (in RAG) to ingest time, and answers read the wiki rather than the raw sources.

Source: A. Karpathy, "LLM Wiki", GitHub gist, April 2026 (gist.github.com/karpathy/442a6bf555914893e9891c11519de94f).

Karpathy's LLM Wiki: continuously growing a KB for QA

A similar approach for skills

Tang, Liyan, et al. "WikiSkill: Compiling agent experience into persistent knowledge for skill evolution." *arXiv preprint arXiv:2608.27454* (2026).

Google Research

2026-08-28

WikiSkill: Compiling Agent Experience into Persistent Knowledge for Skill Evolution

Liyan Tang[1], Cyrus Rashtchian[1], Chun-Sung Ferng[1], Andrew Tomkins[1], Da-Cheng Juan[1] and Tu Vu[1,2]
[1]Google Research, [2]Virginia Tech

Agent skills **package specialized knowledge and workflows into reusable resources that extend AI agent capabilities. Recent work automatically discovers such skills from agent experience, which enables agents to progressively adapt through interaction. However, the insights that guide skill development typically remain scattered across optimization histories, limiting their systematic reuse across iterations. We introduce WikiSkill, a framework that co-evolves agent skills with a *persistent* knowledge base (wiki). At a high level, WikiSkill separates raw execution experience, accumulated knowledge, and executable skills, while continuously consolidating experience into the wiki, which subsequent skill updates can build on. Across diverse benchmarks and models, WikiSkill consistently outperforms *state-of-the-art* skill-evolution methods and improves over no-skill baselines in most model-benchmark settings. We find that skill evolution complements model scaling: larger models generally benefit more from evolved skills, while smaller models with skills can outperform substantially larger models without them. We also find that evolved skills transfer effectively across models and model families, and skills evolved by other models can outperform self-evolved skills. Finally, our ablation studies confirm that persistent knowledge accumulation in the wiki is critical for effective skill evolution. These results demonstrate the benefits of systematically accumulating and refining agent experience for developing reusable and transferable skills.**

Karpathy's LLM Wiki: continuously growing a KB for QA

A similar approach for skills

Tang, Liyan, et al. "WikiSkill: Compiling agent experience into persistent knowledge for skill evolution." *arXiv preprint arXiv:2608.27454* (2026).

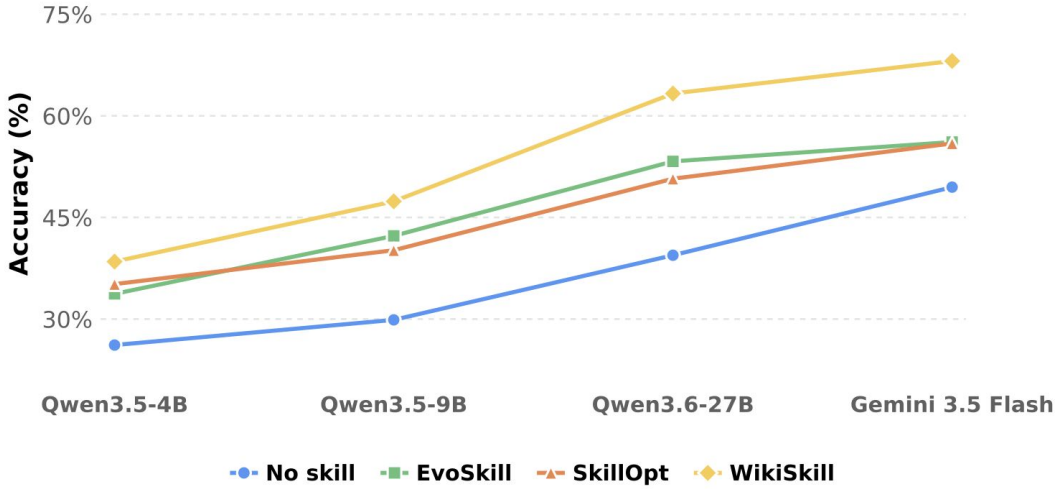


Figure 1 | **WikiSkill consistently improves over both the no-skill baseline and existing skill-evolution methods.** Interestingly, its advantage becomes more pronounced for stronger models. We report average accuracy across the evaluated benchmarks for each model using no skills or skills evolved by EvoSkill, SkillOpt, and WikiSkill (see Table 1 for details).

Karpathy's LLM Wiki: continuously growing a KB for QA

A similar approach for skills

Tang, Liyan, et al. "WikiSkill: Compiling agent experience into persistent knowledge for skill evolution." *arXiv preprint arXiv:2608.27454* (2026).

- Raw sources are execution traces. The raw layer holds immutable execution traces from training rollouts.
- The wiki is very similar and contains what worked and what failed. But, for skills.
- Skill Proposer: It reads the index and the impact file, inspects traces as needed, and produces one atomic proposal per iteration (a new skill or a patch to an existing one).
- The outcome in the wiki are executable skills. Instead of answers, the third layer is skills. Each has a SKILL.md and a PURPOSE.md that links to the wiki patterns that motivated it.

DySECT

Amin-Naseri, M., Kim, H., & Hruschka, E. (2026). A Dynamic Self-Evolving Extraction System. *ACL2026 (Demo).*

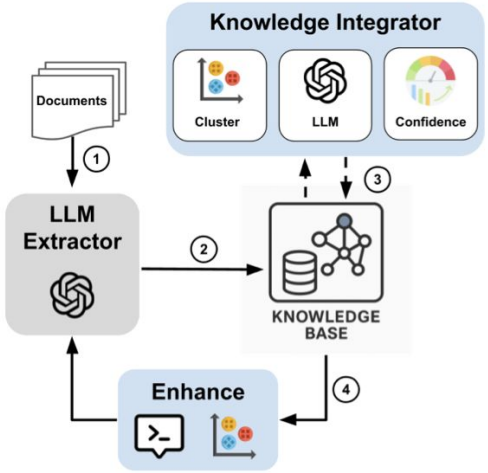


Figure 1: Overview of our closed-loop extraction and knowledge base system. The extractor produces triples from raw text, which populate the KB and are then fed back to improve future extraction.

A Dynamic Self-Evolving Extraction System

Moin Amin-Naseri
Megagon Labs
moin@megagon.ai

Hannah Kim
Megagon Labs
hannah@megagon.ai

Estevam Hruschka
Megagon Labs
estevam@megagon.ai

Abstract

The extraction of structured information from raw text is a fundamental component of many NLP applications, including document retrieval, ranking, and relevance estimation. High-quality extractions often require domain-specific accuracy, up-to-date understanding of specialized taxonomies, and the ability to incorporate emerging jargon and rare outliers. In many domains–such as medical, legal, and HR–the extraction model must also adapt to shifting terminology and benefit from explicit reasoning over structured knowledge. We propose DySECT, a Dynamic Self-Evolving Extraction and Curation Toolkit, which continually improves as it is used. The system incrementally populates a versatile, self-expanding knowledge base (KB) with triples extracted by the LLM. The KB further enriches itself through the integration of probabilistic knowledge and graph-based reasoning, gradually accumulating domain concepts and relationships. The enriched KB then feeds back into the LLM extractor via prompt tuning, sampling of relevant few-shot examples, or fine-tuning using KB-derived synthetic data. As a result, the system forms a symbiotic closed-loop cycle in which extraction continuously improves knowledge, and knowledge continuously improves extraction[1].

named entities, relations, and events across diverse formats (Xu et al., 2024; Zhang et al., 2025). At the same time, domain-specific case studies, for example in scientific and clinical text, show that high-quality extraction still requires careful schema design, substantial domain adaptation, and rigorous evaluation (Dagdelen et al., 2024; Adam, 2025). Taken together, these lines of work suggest that both classic neural IE systems and newer LLM-based approaches remain heavily dependent on curated datasets and manually engineered adaptation strategies.

Despite progress in label-efficient and continual learning methods, updating IE systems in response to new data remains cumbersome. Self-training and bootstrapping approaches reduce annotation costs but typically rely on manually designed seed rules, confidence thresholds, and offline retraining cycles (Angeli et al., 2015; Sarkhel et al., 2023). Domain-adaptive continual learning for LLMs aims to incrementally absorb new domain data without catastrophic forgetting, yet it generally assumes explicit training phases, access to model weights, and non-trivial engineering to maintain stability (Rostami, 2024; Kharrat et al., 2024). Frameworks that combine ontologies or knowledge bases with language models can

DySECT: extraction and the knowledge base improve each other

Amin-Naseri, M., Kim, H., & Hruschka, E. (2026). A Dynamic Self-Evolving Extraction System. *ACL2026 (Demo).*

- Traditional LLM extractors handle each document in isolation and lose what earlier extractions found. That hurts in domains such as HR, legal and medical, where terminology shifts, taxonomies evolve and rare concepts matter.
- In Dysect, very extraction call inserts triples into a self-evolving knowledge base with probabilistic confidence. The KB enriches itself and feeds back into the extractor.
- The system improves through use with no retraining cycle, nor curated adaptation data or access to model weights.
- The loop can run on its own, but an interactive KB interface lets curators inspect, validate or correct what it has learned.

DySECT: extraction and the knowledge base improve each other

Amin-Naseri, M., Kim, H., & Hruschka, E. (2026). A Dynamic Self-Evolving Extraction System. *ACL2026 (Demo).*

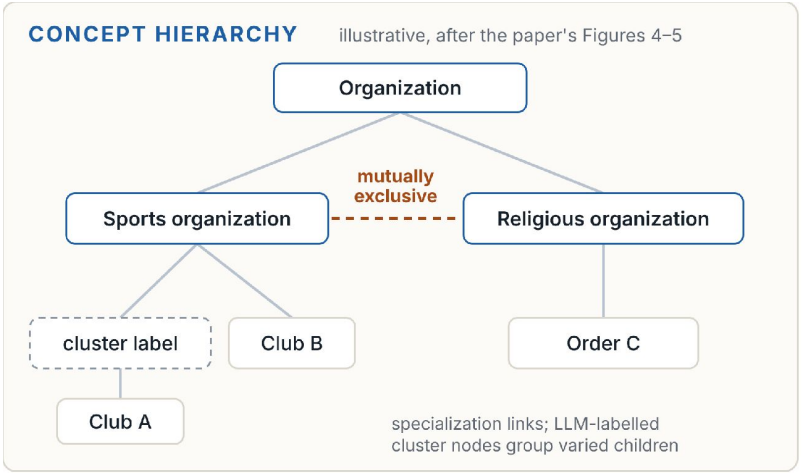


Figure generated with help from Claude Code

DySECT: extraction and the knowledge base improve each other

Amin-Naseri, M., Kim, H., & Hruschka, E. (2026). A Dynamic Self-Evolving Extraction System. *ACL2026 (Demo).*

- Each fact is a (subject, relation, object) triple with typed arguments; inverse relations are added automatically. The KB is built on Theo (Mitchell et al., 1989).
- Local confidences c_i from extractors and acquisition LLMs, a frequency f_i of repeated observations, and the source label of each batch.
- Repeated support combines as $C_agg = 1 - \Pi(1 - \lambda c_i)^{f_i}$, with $\lambda = 0.75$ by default. Each mutually exclusive rival divides it: $C = C_agg / (m + 1)$. Curator decisions are fully trusted ($C = 1.0$).
- Generalization and specialization links, LLM-labelled cluster nodes under broad concepts, and automatically detected mutually exclusive concept families, such as sports versus religious organizations.

DySECT: extraction and the knowledge base improve each other

- The outer loop ingests each batch of extractor output with a source label. An inner loop then runs a few rounds of knowledge integration, concept acquisition and relation acquisition; proposed items enter as candidates with an initial confidence.
- High-confidence subconcepts and representative relations go into the prompt as positive examples, so the extractor finds more facts like them.
- Saturated subconcepts are marked as already extracted, steering the extractor toward relations it has not yet covered.
- Few-shot examples sampled from the KB, or fine-tuning on text generated from high-confidence triples. Neither was used in the reported experiments.
- Through the interface they validate, invalidate or add triples, and track sources and confidence across iterations.

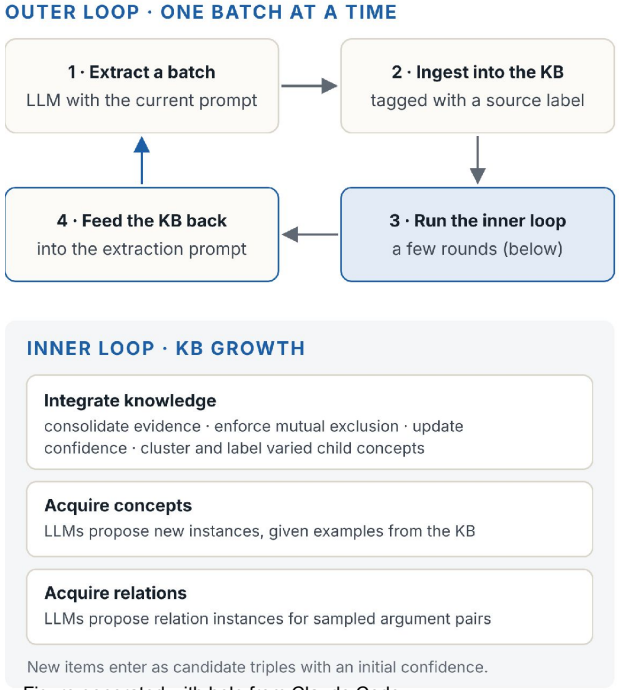
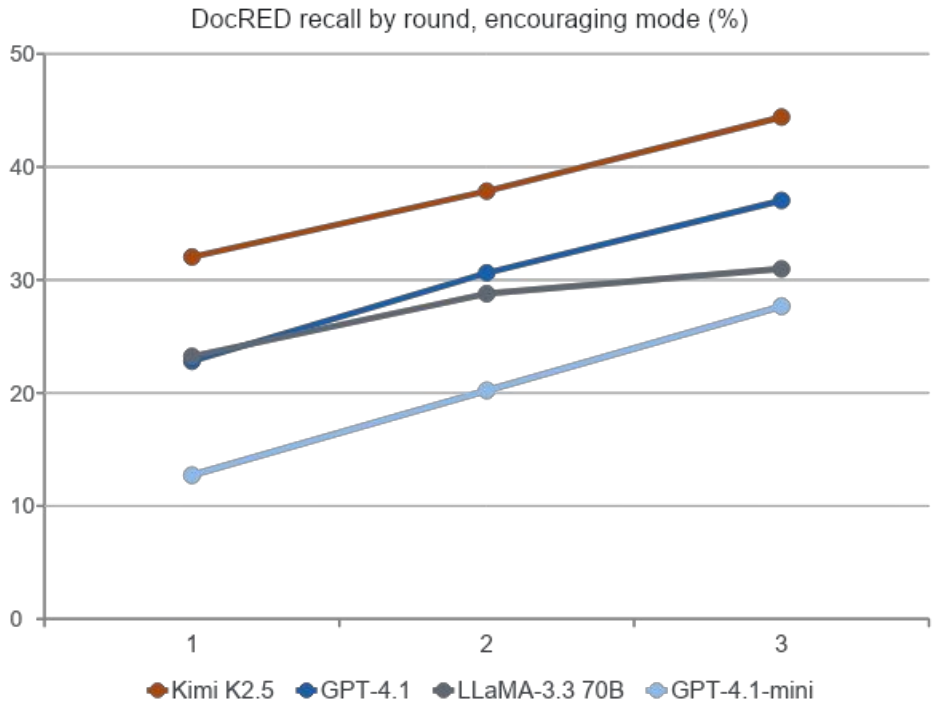


Figure generated with help from Claude Code

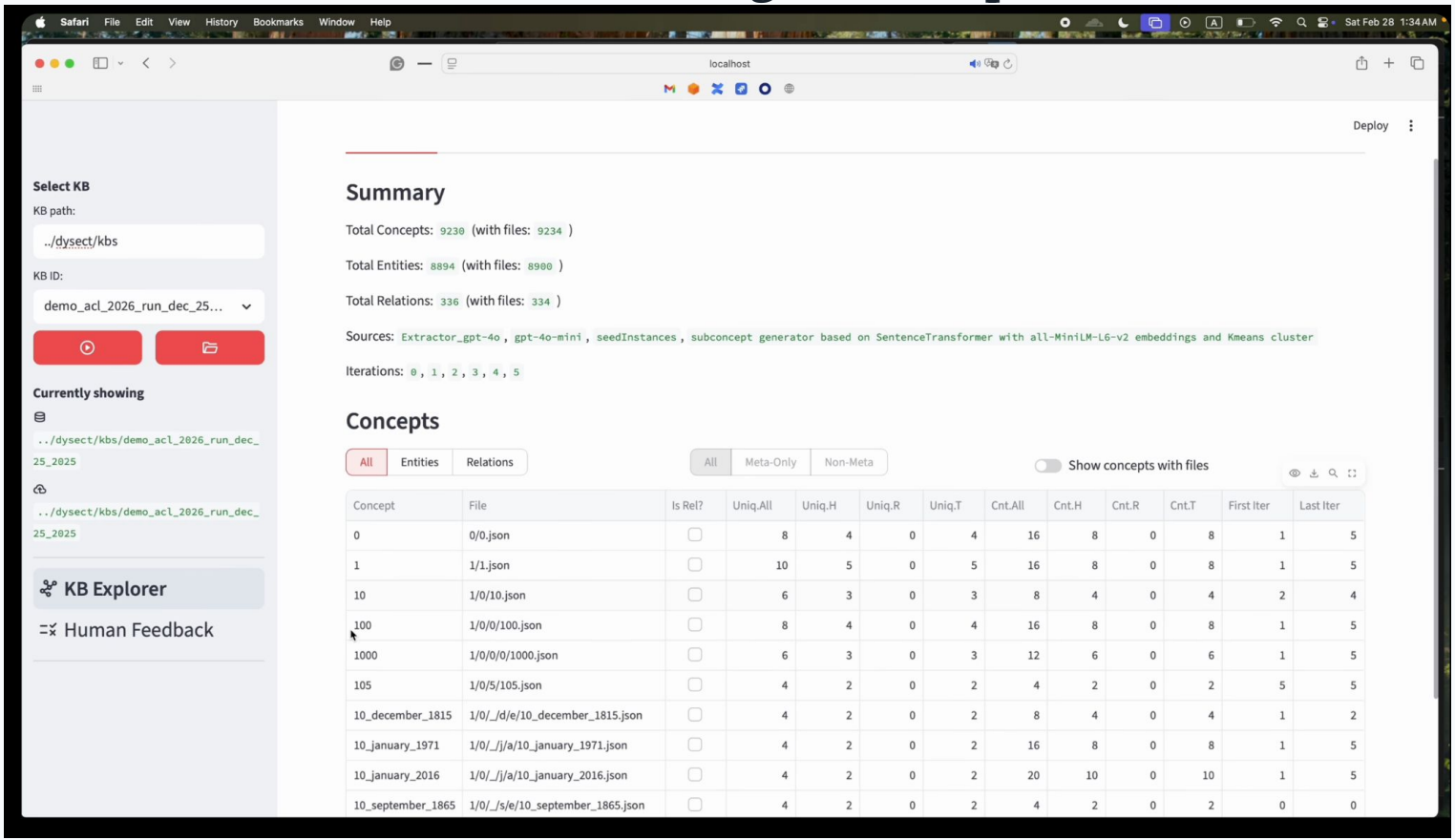
DySECT: extraction and the knowledge base improve each other

Evaluation

- DocRED (96 relation types): 500 fixed documents, a KB initialized from 100 other documents, four extractors, measuring recall and the number of extracted triples. Prompt feedback only: no fine-tuning or synthetic data.
- One round of KB feedback raised recall for every model: +5.6 to +7.8 points in encouraging mode, +4.5 to +5.9 in prohibitive mode. After two rounds GPT-4.1 went from 22.8 to 37.0 (chart). Encouraging mode was more stable; prohibitive mode explored more but added noisier extractions.

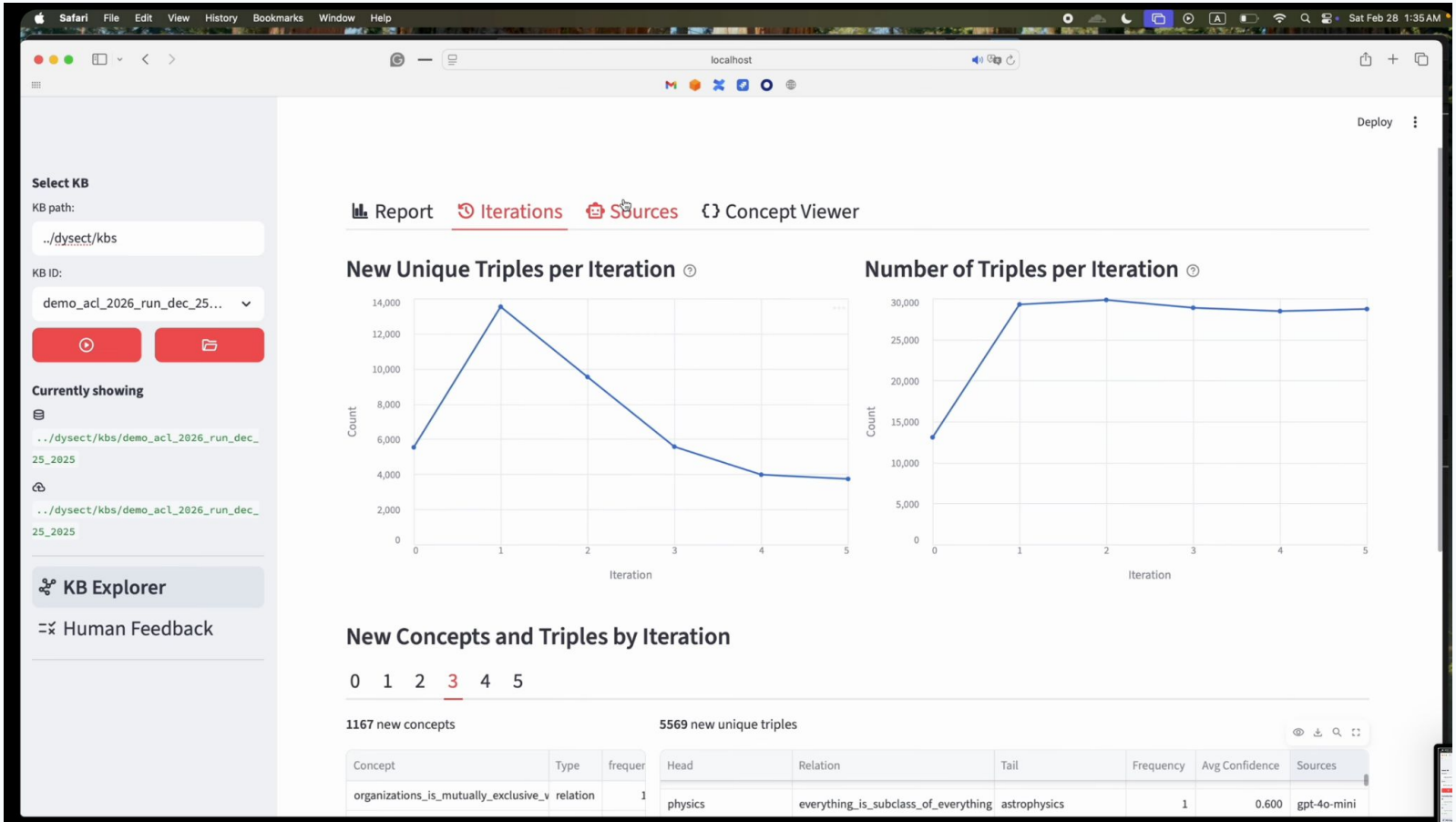


DySECT: extraction and the knowledge base improve each other

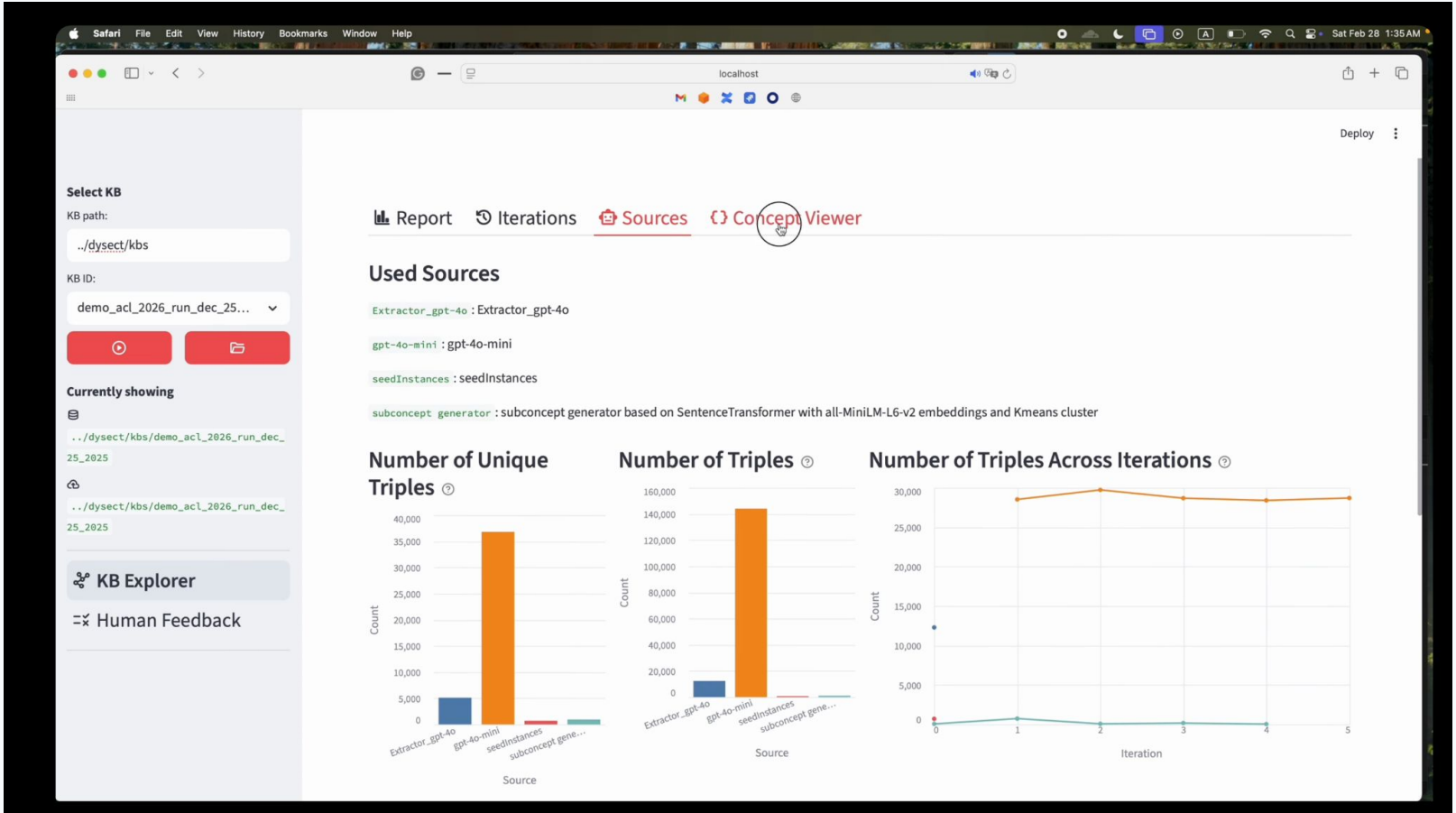


Concept	File	Is Rel?	Uniq.All	Uniq.H	Uniq.R	Uniq.T	Cnt.All	Cnt.H	Cnt.R	Cnt.T	First Iter	Last Iter
0	0/0.json	☐	8	4	0	4	16	8	0	8	1	5
1	1/1.json	☐	10	5	0	5	16	8	0	8	1	5
10	1/0/10.json	☐	6	3	0	3	8	4	0	4	2	4
100	1/0/0/100.json	☐	8	4	0	4	16	8	0	8	1	5
1000	1/0/0/0/1000.json	☐	6	3	0	3	12	6	0	6	1	5
105	1/0/5/105.json	☐	4	2	0	2	4	2	0	2	5	5
10_december_1815	1/0/_/d/e/10_december_1815.json	☐	4	2	0	2	8	4	0	4	1	2
10_january_1971	1/0/_/j/a/10_january_1971.json	☐	4	2	0	2	16	8	0	8	1	5
10_january_2016	1/0/_/j/a/10_january_2016.json	☐	4	2	0	2	20	10	0	10	1	5
10_september_1865	1/0/_/s/e/10_september_1865.json	☐	4	2	0	2	4	2	0	2	0	0

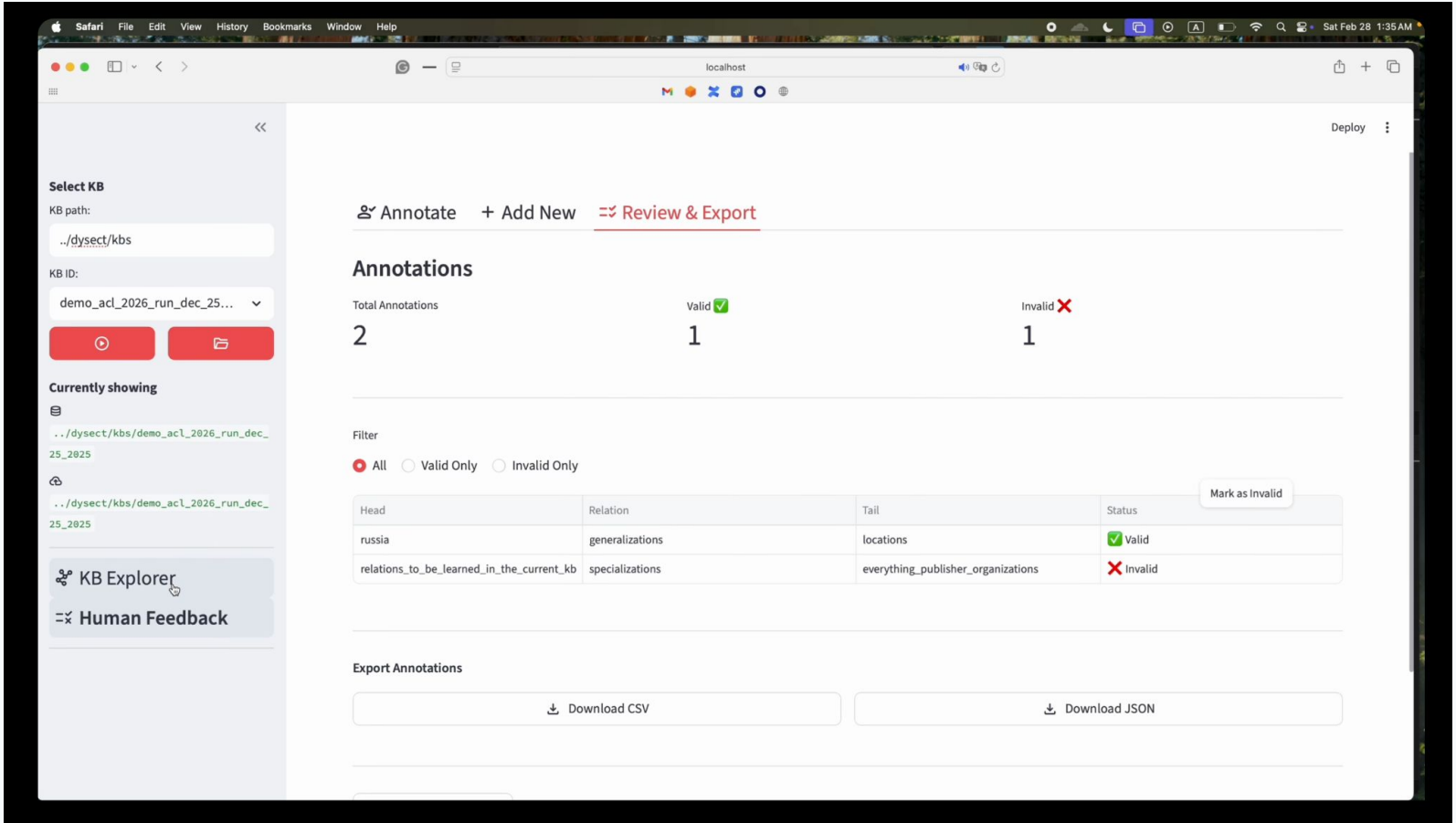
DySECT: extraction and the knowledge base improve each other



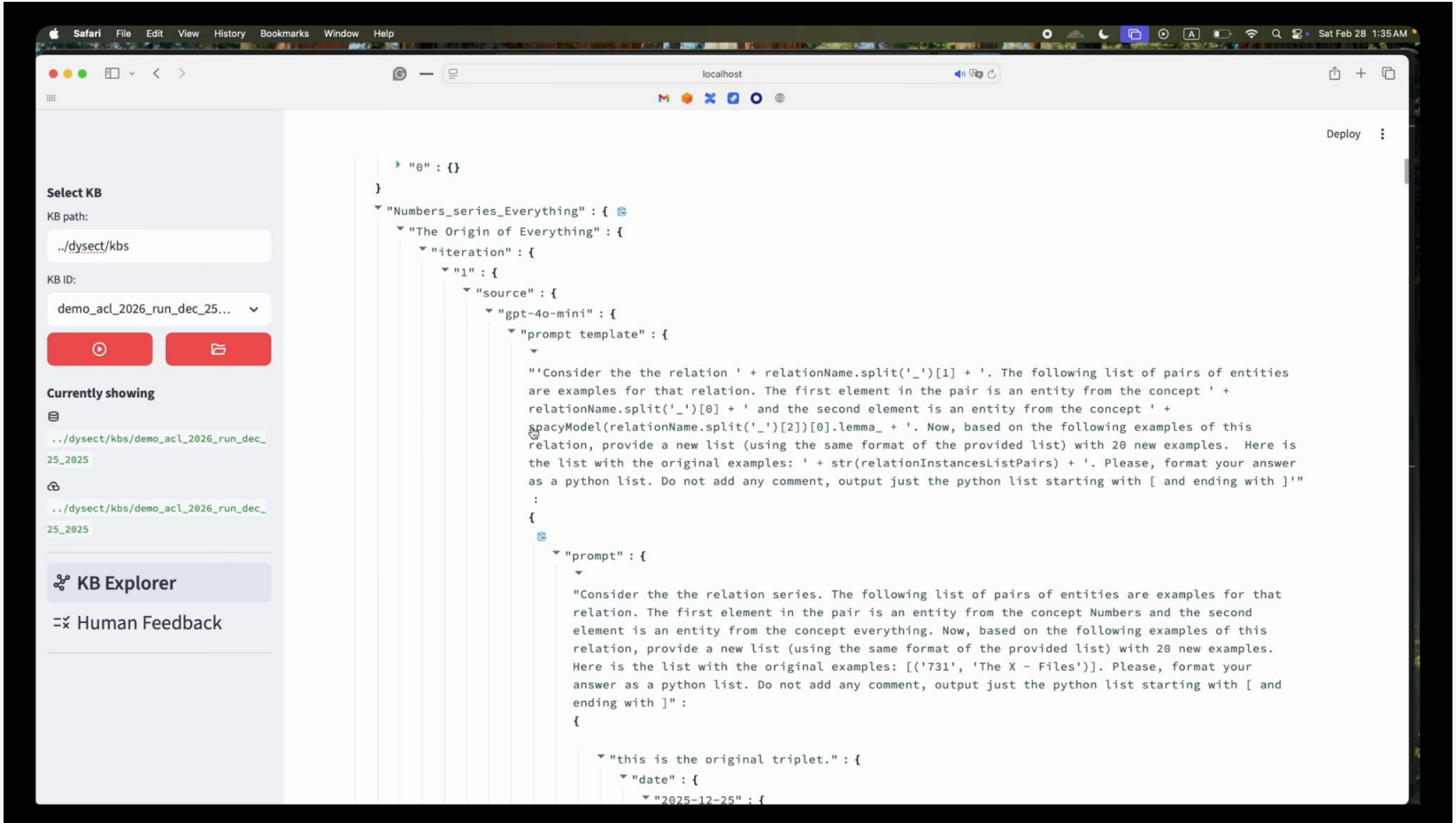
DySECT: extraction and the knowledge base improve each other



DySECT: extraction and the knowledge base improve each other



DySECT: extraction and the knowledge base improve each other



Meta-Harness: optimize the harness, keep the model fixed

Meta-Harness: End-to-End Optimization of Model Harnesses

Yoonho Lee
Stanford

Roshen Nair
Stanford

Qizheng Zhang
Stanford

Kangwook Lee
KRAFTON

Omar Khattab
MIT

Chelsea Finn
Stanford

Pr
Optimized h

Harnes

-artifact

mance

Abstract

The performance of large language model (LLM) systems depends not only on model weights, but also on their *harness*: the code that determines what information to store, retrieve, and present to the model. Yet harnesses are still designed largely by hand, and existing text optimizers are poorly matched to this setting because they compress feedback too aggressively: they are memoryless, condition only on scalar scores, or restrict feedback to short templates or summaries. We introduce **Meta-Harness**, an outer-loop system that searches over harness code for LLM applications. It uses an agentic proposer that accesses the source code, scores, and execution traces of all prior candidates through a filesystem. On online text classification, Meta-Harness improves over a state-of-the-art context management system by 7.7 points while using 4× fewer context tokens. On retrieval-augmented math reasoning, a single discovered harness improves accuracy on 200 IMO-level problems by 4.7 points on average across five held-out models. On agentic coding, discovered harnesses surpass the best hand-engineered baselines on TerminalBench-2. Together, these results show that richer access to prior experience can enable automated harness engineering.

Lee, Y., Nair, R., Zhang, Q., Lee, K., Khattab, O., & Finn, C. (2026). Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.

Meta-Harness: optimize the harness, keep the model fixed

Meta-Harness: End-to-End Optimization of Model Harnesses

Yoonho Lee Stanford
Roshen Nair Stanford
Qizheng Zhang Stanford
Kangwook Lee KRAFTON
Omar Khattab MIT
Chelsea Finn Stanford

Lee, Y., Nair, R., Zhang, Q., Lee, K., Khattab, O., & Finn, C. (2026). Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.

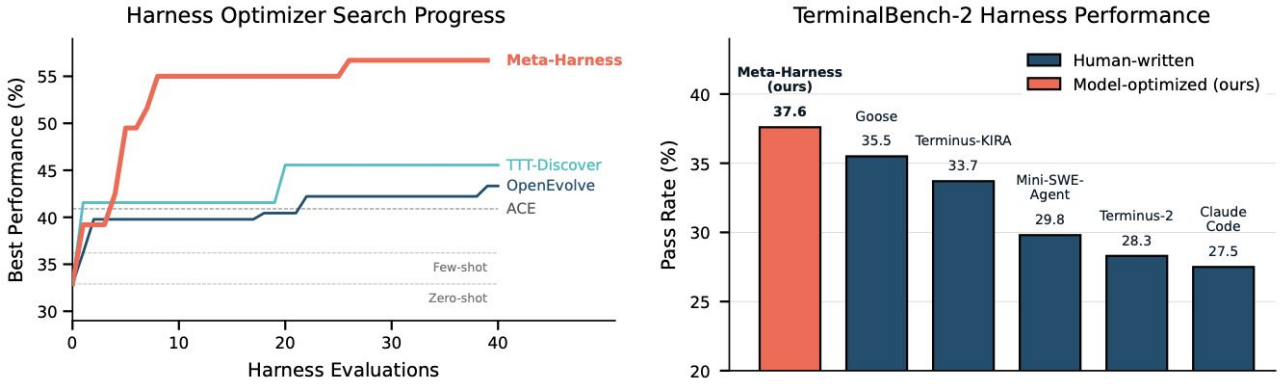


Figure 1: **(Left)** On text classification, Meta-Harness outperforms the best prior hand-designed harnesses (ACE) and existing text optimizers (TTT-Discover, OpenEvolve), matching the next-best method's final accuracy after just 4 evaluations. **(Right)** On TerminalBench-2, Meta-Harness outperforms all reported `Claude Haiku 4.5` harnesses.

Meta-Harness: optimize the harness, keep the model fixed

Meta-Harness: End-to-End Optimization of Model Harnesses

Yoonho Lee
Stanford

Roshen Nair
Stanford

Qizheng Zhang
Stanford

Kangwook Lee
KRAFTON

Omar Khattab
MIT

Chelsea Finn
Stanford

Lee, Y., Nair, R., Zhang, Q., Lee, K., Khattab, O., & Finn, C. (2026). Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*.

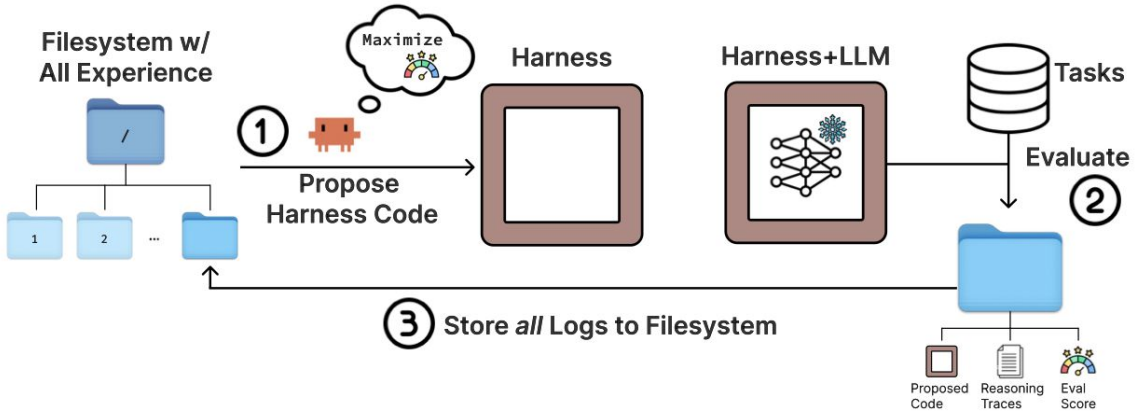


Figure 2: **Meta-Harness search loop. (1)** An agent reads a filesystem containing all prior candidates' source code, execution traces, and scores, and proposes a new harness. **(2)** We evaluate the proposed harness on evaluation tasks. **(3)** All logs (proposed code, reasoning traces, evaluation scores) are stored in the filesystem in a new directory, and the loop repeats.

Meta-Harness: optimize the harness, keep the model fixed

- In this context, Harness is the stateful program around an LLM that decides what context the model sees at each step: what to store, retrieve and show, and how tools are called. Engineers usually tune it by hand.
- Traditional prompt and text optimizers learn from compressed feedback such as scores, summaries or short windows. Harness failures often trace back to decisions made many steps earlier, and summaries hide them.
- In Meta-Harness, an outer loop in which a coding agent (Claude Code with Opus 4.6) reads the code, scores and execution traces of every earlier harness from a filesystem, then writes a better one. The model weights never change.
- The learned artifact is code, so every change is readable, diffable and reversible, and the proposer never sees held-out test results.

Meta-Harness: optimize the harness, keep the model fixed

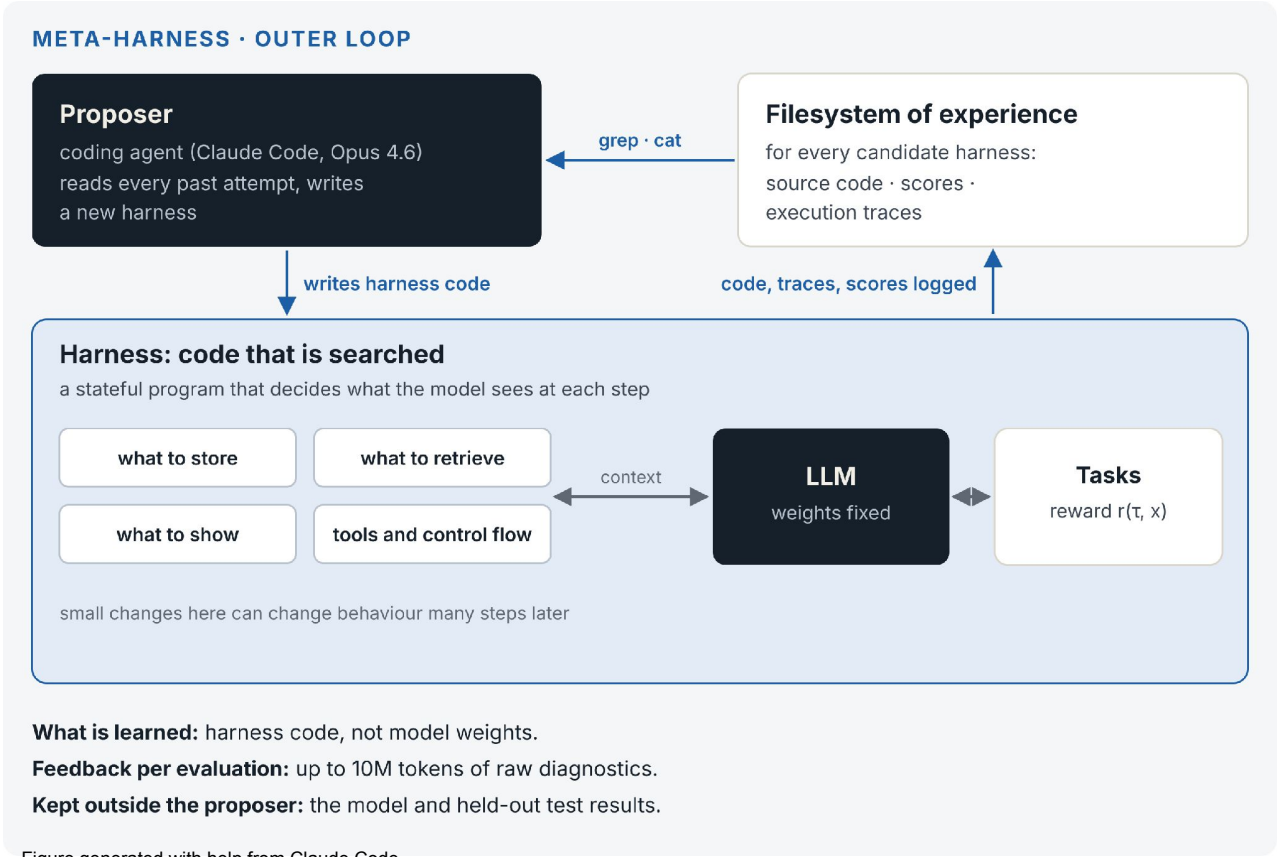


Figure generated with help from Claude Code

Meta-Harness: optimize the harness, keep the model fixed

Figure generated with help from Claude Code

THE PROPOSER'S FILESYSTEM

illustrative layout: one directory per evaluated harness

```
search_run/
├─ SKILL.md                    rules: where to write, what may be modified
├─ cand_001/
│  ├─ harness.py               full source code
│  ├─ scores.json              accuracy, context tokens
│  └─ traces/                  prompts, tool calls, model outputs, state updates
│     ├─ task_0007.log
│     └─ …
├─ cand_002/  …
├─ …
└─ cand_040/  …
```

Read selectively, not all at once

the proposer navigates with grep and cat

82 files median read per iteration

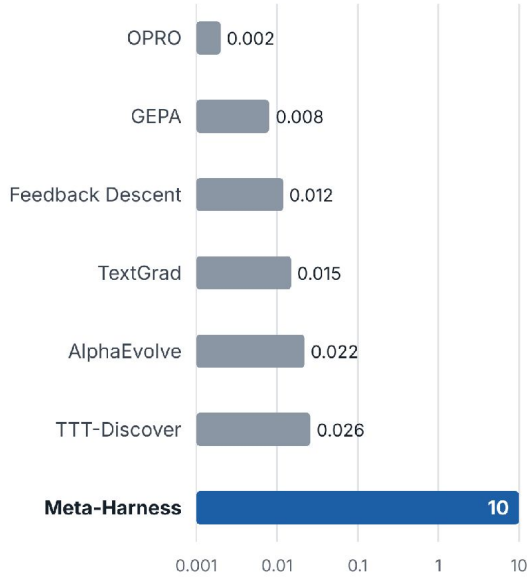
20+ earlier candidates referenced

in the most demanding setting

Figure generated with help from Claude Code

FEEDBACK PER STEP

million tokens per iteration, log scale (paper's Table 1)



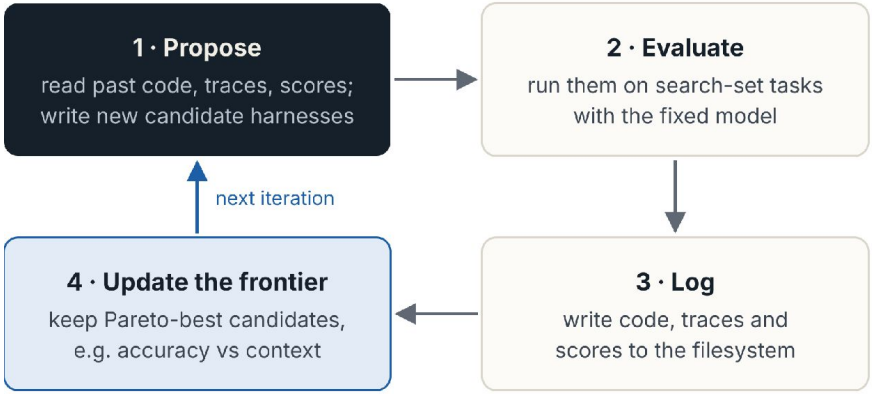
About three orders of magnitude more diagnostic context than prior text optimizers, because nothing is summarized away up front.

Meta-Harness: optimize the harness, keep the model fixed

- Each evaluated harness keeps its source code, its scores, and its execution traces: prompts, tool calls, model outputs and state updates.
- The proposer browses with grep and cat rather than loading everything. In the most demanding setting it read a median of 82 files per iteration, drawing on more than 20 earlier candidates.
- A short skill file says where to write harnesses, how to inspect earlier ones and which files may change. There is no fixed search template or separate memory module: the files are the memory.
- One evaluation can produce up to 10 million tokens of diagnostics, about three orders of magnitude more than prior text optimizers use (right).

Meta-Harness: optimize the harness, keep the model fixed

ONE ITERATION



A TYPICAL RUN

~20 iterations, a few candidates each (2 per iteration for text classification); a few hours of wall-clock time

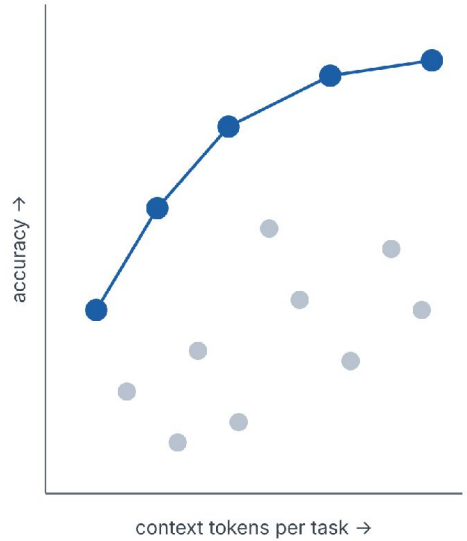
No parent-selection rule: the proposer decides which past candidates to inspect and build on

Example diagnosis: two regressions shared a prompt edit; the proposer isolated the structural change, then tried a safer additive edit, which became the run's best

After the last iteration: test the frontier once on held-out tasks

PARETO FRONTIER

illustrative points; after the paper's Figure 3



- frontier: no other candidate is both more accurate and cheaper
- dominated candidates stay on disk as evidence for later steps

Figure generated with help from Claude Code

Meta-Harness: optimize the harness, keep the model fixed

- Each iteration the proposer writes new candidates (two per iteration for text classification, over about 20 iterations). They run on search-set tasks with the fixed model, and their code, traces and scores go back into the filesystem.
- Candidates are compared on several objectives, for example accuracy against context tokens. There is no hand-written rule for choosing which candidate to build on.
- The proposer never sees test results. After the last iteration, the frontier is evaluated once on held-out tasks.
- In one run the proposer noticed that two regressions shared a prompt edit, isolated the structural change, and switched to a safer additive edit, which became the run's best harness.

Meta-Harness: optimize the harness, keep the model fixed

- Text classification (three datasets, GPT-OSS-120B): 48.6% average accuracy versus 40.9% for ACE, with 11.4K instead of 50.8K context tokens. Best on 6 of 9 unseen datasets (73.1% vs 70.2%).
- Math with retrieval (200 IMO-level problems, five models): +4.7 points over no retrieval, but only +1.3 over plain BM25.
- TerminalBench-2 (89 tasks): 76.4% with Opus 4.6, second among Opus agents; 37.6% with Haiku 4.5, first.
- With full traces the median reached 50.0%, versus 34.9% with summaries and 34.6% with scores only.
- Only one proposer was tested. TerminalBench-2 search and evaluation used the same tasks; the authors audited for leakage. The model used to select the math harness was also among the five evaluated. Repeated selection on a search set can overfit it, though overfitting in code is easier to inspect than in weights.

LLM/Skills wiki, DySECT and Meta-Harness

The LLM and skills wiki, DySECT and Meta-Harness all improve a deployed system without changing model weights. The question is how far each goes toward Post-Deployment Self-Improving agents.

- Does the system keep learning from its own use, couple several learners and grow its task set?
- Does it separate candidate from promoted beliefs, keep explicit state with provenance, and control errors?
- Does it improve the answer path and the acquisition path, with evidence over time?
- Does light human supervision correct drift?

.

LLM and skills wiki: compounding memory, light governance

ACCOMPLISHES

Persistent, compounding memory. Knowledge is compiled once into readable pages and refined with every source and query, instead of being re-derived per question.

Explicit, versioned state. Pages cite sources and live in Git with an index and a log, so every change can be inspected and reverted.

Knowledge that improves learning. In WikiSkill, giving the skill proposer the wiki raised average accuracy from 48.7 to 63.7: the clearest sign among the three cases that accumulated knowledge makes the learner better.

LLM and skills wiki: compounding memory, light governance

FALLS SHORT

Weak belief control. Pages are written without a candidate stage, confidence or constraints between claims, so ingest errors get baked in; WikiSkill never rolls back or prunes its wiki.

Short horizon. One 13-question study and offline benchmark runs; no new learning problems are created.

DySECT

ACCOMPLISHES

- Coupled learners over one KB: Extraction, concept acquisition and relation acquisition share a knowledge base constrained by hierarchies and mutual exclusion, as in NELL.
- Confidence-weighted beliefs with provenance: Each triple keeps confidences, frequency and source labels; conflicts lower confidence; curator decisions are trusted.
- A growing ontology and measured gains: New subconcepts and exclusive families emerge, and one round of KB feedback raised DocRED recall for all four extractors (+5.6 to +7.8 points).

DySECT

FALLS SHORT

Early evidence: Two rounds on one dataset, recall only (precision not reported); faster learning of new relations is untested.

Meta-Harness: rigorous promotion, short horizon

ACCOMPLISHES

- Candidate versus promoted: Every candidate is scored on a search set, a Pareto frontier is kept, and held-out tasks are used once, at the end.
- Self-diagnosis from raw experience: Reading code, traces and scores lets the proposer explain regressions; with raw traces the median result was 50.0%, against about 35% with summaries.
- Learning stored as code: The learned artifact is versioned, inspectable code, with large gains in text classification and transfer to unseen datasets.

Meta-Harness: rigorous promotion, short horizon

FALLS SHORT

- One task, offline: Each run optimizes one harness for one domain over a few hours: no coupled tasks, no growing task set, no learning during deployment.

- Selection pressure: Repeated selection on one search set can overfit it; only one proposer was tested; faster learning over time is not measured.

Broader View

	LLM and skills wiki	DySECT	Meta-Harness
What is learned	Wiki pages; skills in WikiSkill	Typed triples with confidence; a growing ontology	Harness code
Experience used	Sources and queries; agent traces in WikiSkill	Its own extractions, LLM acquisitions, curator edits	Code, traces and scores of every candidate
Coupling	Wiki feeds skills; claims unconstrained	Several learners; hierarchy and mutual-exclusion constraints	None: one harness per domain
Gate	Lint for pages; validation score for skills	Confidence; optional trusted curator	Search-set Pareto frontier; held-out test once
Best evidence	Preregistered 13-question test; WikiSkill ablation	DocRED recall, four extractors, two rounds	Three domains; raw-trace ablation
PDSI strength	Compounding, readable memory	Coupled, confidence-weighted beliefs	Rigorous promotion and self-diagnosis
Main PDSI gap	never-pruned knowledge	No precision or long-run evidence	Offline, single-task search

What a PDSI agent would take from each

- Complementary strengths: Readable, compounding memory from the LLM Wiki; an evidence layer feeding gated, reversible skills from WikiSkill; coupled learners and confidence-weighted beliefs from DySECT; trace-based diagnosis and held-out promotion from Meta-Harness.
- Shared gaps (shaded): None runs for long during deployment, whereas PDSI should work every day for years.
- Next step: Combine DySECT-style beliefs, wiki-style memory and Meta-Harness-style, and evaluate the result longitudinally with the Part V protocol. Part VII turns these gaps into open problems.

KEY MESSAGE · PART VI

Different substrates, same obligations: candidate → evidence → gate → versioned commit → longitudinal evaluation.

Continuous learning is useful when it is selective, verifiable and reversible.

Next · Part V — Design Patterns

open research problems

Diagnosis-driven task invention

Learning coupling structure

Separate information gain from population shift

Evaluator co-evolution

Long-horizon reproducibility

Learning to learn, measured

Questions for the *ACL community

Benchmarking self-improvement. Streams with fixed targets and evolving tasks, where adding easy tasks cannot inflate progress (sketch, right).

Multilingual and domain adaptation. Methods built on English and high-resource benchmarks may not transfer, and feedback loops can widen gaps between languages.

Online versus offline. Which updates belong in deployment, and which in a curated offline cycle? All three case studies learn offline or between rounds.

Reward hacking and proxies. How to keep an agent from optimizing its judge instead of its task. Cursor’s real-time RL already met this in production.

Discussion: which of these would you put in a shared task?

A design and evaluation checklist for self-improving agents

Design commitment	Evidence or specification required
Target tasks	Inputs, outputs, populations, metrics, protected constraints
Learner boundary	Persistent state, transient context, external dependencies
Experience	Sources, selection process, supervision, exposure history
Retained pathway	What changes and how it can affect future decisions
Views	Complementarity, provenance, dependence assumptions
Coupled tasks	Meaning and permissions of each learning edge
New tasks	Diagnosis, specification, alternatives, target transfer test
Substrates	Scope, generalization, cost, auditability, reversibility
Governance	Proposal, assessment, authorization, commitment, monitoring
Repair	Version bundles, descendants, rollback, compensation
Evaluation	Frozen and updating comparisons, prospective tests, uncertainty
Reporting	Gains, failures, resources, limitations, reproducible history

The conclusion: conditional and testable

- A deployed language agent can improve through retained changes across many substrates: knowledge bases, wikis, skills, harness code and weights.

- That improvement is established only when future performance, learning efficiency or preservation under change improves under a defensible comparison.

- What makes it credible is evidence, governance, revision and longitudinal evaluation.

Where to read next

- Start with the foundations: Mitchell et al. define never-ending learning; Silver and Sutton argue for agents that learn from their own stream of experience.

- Add the two literatures this tutorial connects. Continual learning of LLMs, which Hou et al. describe as moving from weights to memory, skills and protocols; and self-evolving agents with persistent memory.

- Check also related tutorials. Two 2025 tutorials treat continual learning of LLMs mainly at training time, one at IJCNLP-AACL 2025. This tutorial adds the post-deployment, governed, multi-substrate view.